

Accessibility Assessment Scheme Tool (AAST)

Backend Developer Guide

EXPRESS · TYPESCRIPT · MARIADB · EN 17210

Version: 1.0 | **Release:** 2026-06-17 | **Document ID:** AAST-DOC-BACKEND-001

© 2026 AAST. Internal — Backend Engineering Team.



1 EXECUTIVE SUMMARY

The AAST Backend (the "Kernel") is a metadata-driven REST API implementing the **EN 17210** accessibility scoring framework. It is the single source of truth for all business logic, calculations, and data access.

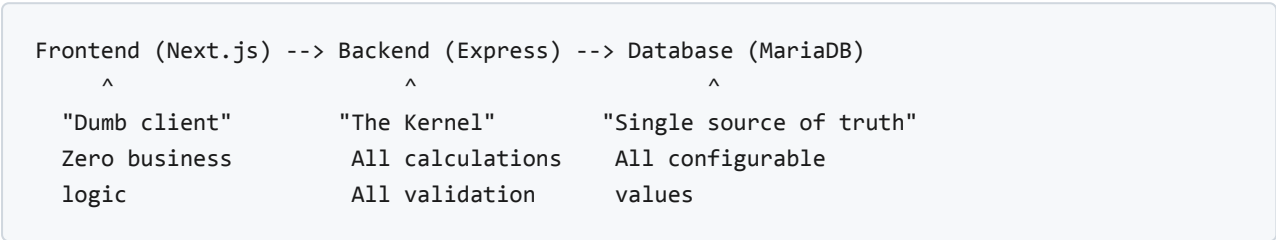
RESPONSIBILITY	IMPLEMENTATION
Calculation Engine	5-layer EN 17210 algorithm (IS → TIS → CIS → OBS → NEB)
Data Persistence	TypeORM + MariaDB with 5 entities
REST API	14 endpoints across 6 controllers
Validation	Zod schemas for all POST/PUT
Caching	Lazy-initialized in-memory metadata cache
Security	Helmet, CORS, HTTPS, TLS 1.2
Enforcement	Static test suite bans hardcoded patterns

Core principle: All configurable values live in the database — weights, thresholds, criteria, and configuration. Domain experts update the system via SQL or API. No code changes. No deployments.

COMPONENT	TECHNOLOGY
Runtime	Node.js 20 LTS
Framework	Express.js 4.18
Language	TypeScript 5.1+ (ES2022, CommonJS)
ORM	TypeORM 0.3.17
Database	MariaDB 10.11
Validation	Zod 4.4+
Containerization	Docker Compose
Cloud	Azure (Bicep IaC)
CI/CD	GitHub Actions

2 PROJECT OVERVIEW

Architecture



Key Architectural Decisions

1. **Metadata-Driven:** All configurable values in the database, never in code
2. **Single Source of Truth:** Backend is sole authority for calculations
3. **Dynamic Type Detection:** DT 1–5 and AD 1–5 derived from criteria data
4. **Config-Driven Thresholds:** NEB breakpoints, strength/weakness, MAX_OBS are all DB rows
5. **Lazy Metadata Caching:** Initializes once, caches lookup data in memory
6. **Static Test Enforcement:** CI blocks deployment if hardcoded patterns detected

Business Purpose

The AAST evaluates buildings against **63 canonical criteria** across a 5×5 matrix:

- **5 Disability Types (DT):** Physical, Sensory, Cognitive & Neurodiverse, Communication & Mental Health, Multiple/Situational
- **5 Assessment Dimensions (AD):** Spatial & Physical, Safety & Environmental, Cognitive & Navigational, Digital Interaction, Social Inclusion

Each criterion has a unique code `EC{DT}.{AD}.{index}` (e.g., `EC1.1.1`).

Certification flow: Assessor selects building type → rates criteria 1–5 → Backend computes weighted OBS% → Maps to NEB class (A+/A/B/No Rating) → Returns strengths, weaknesses, per-DT/AD scores → Persists with full audit trail.

Targets compliance with **EN 17210** and EU accessibility regulations.

3 SYSTEM ARCHITECTURE

```
+-----+
|                PRESENTATION LAYER                |
| Next.js Pages --> React Components --> Custom Hooks --> Axios |
+-----+
|                API GATEWAY                        |
| Express Router -> Helmet -> CORS -> JSON Parser -> Error Handler |
+-----+
|                BUSINESS LOGIC LAYER              |
| Calculation Service (5-layer engine) + Metadata Service |
| Zod Validation                                         |
+-----+
|                DATA ACCESS LAYER                |
| TypeORM Repositories --> Entities --> Migrations      |
+-----+
|                PERSISTENCE LAYER                 |
|                MariaDB 10.11                      |
+-----+
```

Data flow: User Input → Hook → Axios POST → Router → Controller → Service → Repository → MariaDB → Response → React Render

4 BACKEND OVERVIEW

Folder Structure

```
backend/
├─ docker-compose.yml          # MariaDB container
├─ docker-initdb.d/init.sql    # DB user/database creation
├─ package.json / tsconfig.json
├─ src/
│   ├─ index.ts                # Express entry point
│   ├─ data-source.ts          # TypeORM DataSource config
│   └─ controllers/            # 6 route handlers
│       ├─ evaluateController.ts # POST /api/v1/evaluate
│       ├─ dataController.ts    # GET/POST criteria, types, thresholds, config
│       ├─ buildingsController.ts # GET buildings
│       ├─ assessmentsController.ts # POST/GET assessments
│       ├─ reportsController.ts  # GET reports
│       └─ authController.ts     # POST /api/v1/auth/login
│   └─ services/
│       ├─ calculationService.ts # 5-layer EN 17210 engine
│       └─ metadataService.ts    # Lazy cache + validation
│   └─ entities/                # Criterion, BuildingType, Evaluation, NebThreshold,
Config
├─ migrations/                  # 3 sequential schema migrations
├─ data/                        # Seed data (criteria.ts, building-types.json)
├─ scripts/                     # seed.ts, runMigrations.ts, workbook scripts
└─ tests/                       # calculation.test.ts, static.test.ts, metadata.test.ts
```

Controllers

Controllers handle request parsing, validation, and response formatting. **No business logic** — they delegate to services.

CONTROLLER	KEY ENDPOINTS	PURPOSE
<code>evaluateController</code>	<code>POST /evaluate</code>	Full EN 17210 evaluation
<code>dataController</code>	<code>GET/POST /criteria</code> , <code>GET /building-types</code> , <code>GET /neb-thresholds</code> , <code>GET /config</code>	Dynamic data CRUD
<code>buildingsController</code>	<code>GET /buildings</code> , <code>GET /buildings/:id</code>	Building directory
<code>assessmentsController</code>	<code>POST /assessments</code> , <code>GET /assessments/:id</code>	Assessment persistence
<code>reportsController</code>	<code>GET /reports</code>	Evaluation history
<code>authController</code>	<code>POST /auth/login</code>	Mock JWT login

Services

Calculation Service — Implements the 5-layer scoring algorithm (see Section 6).

Metadata Service — Lazy-initialized cache:

- Initializes DataSource connection once
- Caches NEB thresholds (ordered DESC by `min`)
- Caches config values per key
- Provides `getBuildingType()`, `getAllCriteria()`, `validateMetadata()`
- Validates all 4 core tables have data on startup; exits code 1 if not

Static Test Enforcement

`static.test.ts` greps ALL source files for banned hardcoded patterns:

-  `0.2,0.2,0.2,0.2,0.2` — equal weights must be DB-driven
-  `85, 60, 40` — NEB thresholds must be in `neb_thresholds` table
-  `MAX_OBS = 25` — config must be in `config` table

This runs in CI and blocks deployment if violated.

5 DATABASE OVERVIEW

Entity-Relationship

```
erDiagram
    building_types {
        VARCHAR_36 id PK
        VARCHAR_255 name UK
        JSON disability_weights
        JSON dimension_weights
    }
    criteria {
        VARCHAR_36 id PK
        VARCHAR_255 code UK
        TEXT name definition justification
        FLOAT value
        INT disability dimension
        INT score
        JSON levels
    }
    evaluations {
        VARCHAR_36 id PK
        VARCHAR_255 user_id building_type
        JSON scores result
        DATETIME created_at
        DOUBLE obs
        VARCHAR neb_class
        DOUBLE average_raw_score
    }
    neb_thresholds {
        VARCHAR_36 id PK
        INT min
        VARCHAR_50 neb_class
        VARCHAR_50 equivalent
        TEXT meaning
        INT neb_score ordinal
    }
    config {
        VARCHAR_36 id PK
        VARCHAR_255 key UK
        TEXT value
    }
    evaluations ||--o{ building_types : "building_type refs name"
```

Table Details

building_types (7 rows)

Stores per-type weight matrices for DT and AD.

COLUMN	TYPE	DESCRIPTION
name	VARCHAR(255) UNIQUE	e.g., "Commercial Buildings"
disability_weights	JSON	5 floats for DT 1–5
dimension_weights	JSON	5 floats for AD 1–5

Example: { "name": "Commercial Buildings", "disability_weights": [0.28,0.22,0.18,0.12,0.2], "dimension_weights": [0.30,0.25,0.18,0.12,0.15] }

criteria (63 rows)

Canonical evaluation criteria. Each has value (importance weight 0–1), disability and dimension classification (1–5), default score (3), and levels (array of 5 descriptors). Example: EC1.1.1 — "Accessible Entrance Width" — value 0.7, DT 1, AD 1.

evaluations (variable rows)

One row per assessment. Stores scores (code→score map), computed result (full JSON), obs (%), neb_class, average_raw_score.

neb_thresholds (4 rows)

MIN	NEB_CLASS	MEANING
85	A+	Excellent / Best practice
60	A	Good
40	B	Acceptable
0	No Rating	Non-compliant

`config` (5 rows)

KEY	VALUE	DESCRIPTION
<code>MAX_OBS</code>	25	Maximum OBS raw score
<code>DEFAULT_SCORE</code>	3	Fallback score
<code>STRENGTH_THRESHOLD</code>	5	Min score for strength
<code>WEAKNESS_THRESHOLD</code>	2	Max score for weakness
<code>TOP_N_RESULTS</code>	5	Results to return

6 CALCULATION ENGINE

Implements the **EN 17210 accessibility scoring framework** — purely metadata-driven, no hardcoded formulas.

The 5-Layer Algorithm

Layer 1: IS	Layer 2: TIS	Layer 3: CIS	Layer 4: OBS	Layer 5: NEB
Impact Score	Total Impact	Combined Impact	Overall Building	
Classification				
per criterion	Score per DT	Score per AD	Score %	Lookup
IS = value x score	TIS = sum IS per DT	CIS = sum(dtW x IS)	OBS = sum(dtWxTIS)	Find 1st
threshold		per AD	/MAX_OBS x 100	where
OBS >= min				

Layer 1: IS (Impact Score)

$IS(c) = c.value \times c.score$ — rounded to 4 decimal places.

Layer 2: TIS (Total Impact Score)

$TIS(dt) = \sum IS(c)$ for all criteria where $c.disability == dt$. DT IDs are **derived from data** — not hardcoded.

Layer 3: CIS (Combined Impact Score)

$CIS(ad) = \sum (dtWeight[dt] \times TIS(dt))$ for all criteria where $c.dimension == ad$.

Layer 4: OBS (Overall Building Score)

$OBS_raw = \sum (dtWeight[dt] \times TIS(dt))$ for all DTs. $OBS_pct = (OBS_raw / MAX_OBS) \times 100$. MAX_OBS is read from `config` table (value: 25). Rounded to 2 decimal places.

Layer 5: NEB Classification

Thresholds loaded DESC by `min`: 85, 60, 40, 0. Find first where $OBS\% \geq threshold.min \rightarrow A+ (\geq 85)$, A (≥ 60), B (≥ 40), No Rating (< 40).

Additional Computations

- **Strengths/Weaknesses:** Top N where score ≥ STRENGTH_THRESHOLD or ≤ WEAKNESS_THRESHOLD (from config)
- **Per-DT/AD averages:** Arithmetic mean of scores grouped by DT/AD

Metadata-Driven Design

Every configurable value lives in the database, never in code:

HARDCODED (X)	METADATA-DRIVEN (✓)
<code>const NEB = [85, 60, 40]</code>	<code>SELECT * FROM neb_thresholds ORDER BY min DESC</code>
<code>const MAX_OBS = 25</code>	<code>SELECT value FROM config WHERE key='MAX_OBS'</code>
Fixed DT/AD arrays	Derived from <code>criteria.disability</code> / <code>.dimension</code>
Redeploy to change	Update DB row — instant effect

Key guarantees: DT/AD sets derived from data; weights per building type from DB; NEB thresholds are DB rows; MAX_OBS is configurable; static tests enforce compliance.

7 API ARCHITECTURE

- **Base URL:** `/api/v1`
- **Response Envelope:** `{ ok: boolean, data?: any, error?: string }`
- **Status Codes:** 200, 201, 400, 404, 409, 500

Endpoint Reference

Evaluation

METHOD	PATH	AUTH	DESCRIPTION
POST	<code>/evaluate</code>	No	Full EN 17210 computation

Request: `{ "buildingType": "Commercial Buildings", "scores": { "EC1.1.1": 5 } }`

Response: `{ ok: true, result: { obs, nebClass, nebMeaning, nebScore, averageRawScore, avgRawScoreByDT, avgRawScoreByAD, strengths, weaknesses, tisByDT, cisByAD, criteria } }`

Data CRUD

METHOD	PATH	DESCRIPTION
GET	<code>/criteria</code>	All 63 criteria
GET	<code>/criteria/:code</code>	Single criterion
POST	<code>/criteria</code>	Add criterion dynamically
GET	<code>/building-types</code>	7 building types with weights
GET	<code>/neb-thresholds</code>	NEB classification thresholds
GET	<code>/config</code>	All config key-values
GET	<code>/disability-types</code>	5 DT labels
GET	<code>/assessment-dimensions</code>	5 AD labels

Assessments & Reports

METHOD	PATH	DESCRIPTION
GET	<code>/buildings</code>	All buildings (joined with evaluations)
GET	<code>/buildings/:id</code>	Building detail
POST	<code>/assessments</code>	Create & save assessment
GET	<code>/assessments/:id</code>	Retrieve saved assessment
GET	<code>/reports</code>	Evaluation history

Auth

METHOD	PATH	DESCRIPTION
POST	<code>/auth/login</code>	Mock JWT login

Dynamic Criteria System

Adding a criterion is a single API call or SQL INSERT — no code changes, no deployment:

```
curl -X POST http://localhost:4000/api/v1/criteria \
  -H "Content-Type: application/json" \
  -d
'{"code":"EC1.1.10","name":"...", "definition":"...", "value":0.15, "disability":1, "dimension":1, "levels":["L1", "L2", "L3", "L4", "L5"]}'
```

Validation rules: `code` must match `/^EC\d+\.\d+\.\d+$/`, `value` 0–1, `disability` / `dimension` 1–5, `levels` exactly 5 strings, duplicate code → 409. The next evaluation automatically includes the new criterion.

8 REQUEST LIFECYCLE

Evaluation Request Flow

```
Browser -> Next.js Hook -> Axios POST /api/v1/evaluate
  -> Express: Helmet -> CORS -> JSON parse
  -> evaluateController: validates body shape
    -> calculationService.evaluate(buildingType, scores)
      -> Read MAX_OBS, thresholds, weights from metadata cache
      -> Load all 63 criteria, apply scores
      -> Compute IS -> TIS -> CIS -> OBS -> NEB
      -> Compute strengths, weaknesses, averages
      -> Return result object
    -> Controller: res.json({ ok: true, result })
  -> Express: 200 JSON response
  -> Axios: unwrap result
  -> React: re-render with OBS gauge, NEB class, strengths, weaknesses
```

Controller Validation

`evaluateController.ts` checks `buildingType` is string/undefined and `scores` is object/undefined. Returns 400 with specific message on failure.

Response Object Shape

`obs`, `nebClass`, `nebEquivalent`, `nebMeaning`, `nebScore`, `averageRawScore`, `avgRawScoreByDT`, `avgRawScoreByAD`, `strengths` (top 5), `weaknesses` (top 5), `tisByDT`, `cisByAD`, `criteria` (all 63 with computed IS).

9 LOCAL DEVELOPMENT SETUP

Prerequisites

TOOL	MIN VERSION	CHECK
Node.js	18 LTS (20 rec.)	<code>node --version</code>
npm	9+	<code>npm --version</code>
Docker	24+	<code>docker --version</code>

Quick Start

```
# 1. Clone and install
git clone <repo-url> && cd aast-project/backend && npm install

# 2. Start MariaDB
docker compose up -d mariadb
# Wait ~10s, verify: docker compose ps

# 3. Seed database
npx ts-node src/scripts/seed.ts
# Output: "DataSource initialized" / "Seeding complete"

# 4. Start backend
npm start
# Output: "Backend listening on 4000"

# 5. Verify
curl http://localhost:4000/health          # -> {"status":"ok"}
curl http://localhost:4000/api/v1/criteria # -> 63 criteria
```

Docker Environment

MariaDB only runs in Docker; backend/frontend run natively.

PROPERTY	VALUE
Image	<code>mariadb:10.11</code>
Host Port	<code>3306</code>
Database	<code>mydb</code>
User/Pass	<code>myuser</code> / <code>mypassword</code>
Root Pass	<code>rootpass</code>
Data Volume	<code>db_data</code>

Environment Variables

VARIABLE	DEFAULT
<code>PORT</code>	<code>4000</code>
<code>DATABASE_URL</code>	<code>mysql://myuser:mypassword@127.0.0.1:3306/mydb</code>
<code>DB_TYPE</code>	<code>mariadb</code>
<code>TYPEORM_SYNCHRONIZE</code>	<code>false</code>

NPM Scripts

SCRIPT	PURPOSE
<code>npm start</code>	Start backend
<code>npm test</code>	Run calculation tests
<code>npm run test:automated</code>	Full suite (metadata + static + calculation)
<code>npm run seed</code>	Seed database
<code>npm run migrate</code>	Run migrations
<code>npm run verify:workbook</code>	Verify Excel workbook mapping

Setup Troubleshooting

SYMPTOM	FIX
ECONNREFUSED :3306	<code>docker compose up -d mariadb</code>
Access denied	Check DATABASE_URL matches docker-compose.yml
Table doesn't exist	<code>npx ts-node src/scripts/seed.ts</code>
Metadata validation failed	Run seed script
Port 4000 in use	<code>netstat -ano findstr 4000</code> then kill PID

10 RUNNING THE PROJECT

Manual Startup (Windows)

Terminal 1 — MariaDB: `cd backend && docker compose up -d mariadb`

Terminal 2 — Backend: `cd backend && npm start`

Terminal 3 — Frontend (optional): `cd frontend && npm run dev`

Dev Shell Scripts

SCRIPT	PURPOSE
<code>start-dev.sh</code>	Start all services
<code>stop-dev.sh</code>	Stop all services
<code>status-dev.sh</code>	Check service status
<code>logs-dev.sh</code>	View service logs

Quick Verification

```
curl localhost:4000/health    # -> {"status":"ok"}

curl -X POST localhost:4000/api/v1/evaluate \
  -H "Content-Type: application/json" \
  -d '{"buildingType":"Commercial Buildings"}'    # -> full result JSON
```

11 COMMON WORKFLOWS

1. Add a New Criterion

```
curl -X POST localhost:4000/api/v1/criteria -H "Content-Type: application/json" \
-d
'{"code":"EC1.1.10","name":"...", "definition":"...", "value":0.15, "disability":1, "dimension":1, "levels":["L1", "L2", "L3", "L4", "L5"]}'
```

Or via SQL:

```
INSERT INTO criteria (id, code, name, definition, justification, value, disability,
dimension, score, levels)
VALUES (UUID(), 'EC1.1.10', '...', '...', '...', 0.15, 1, 1, 3,
'["L1", "L2", "L3", "L4", "L5"]');
```

2. Adjust NEB Thresholds

```
UPDATE neb_thresholds SET min = 65 WHERE neb_class = 'A';
INSERT INTO neb_thresholds VALUES (UUID(), 95, 'A++', '5+', 'Outstanding', 5, 0);
```

3. Create a Building Type

```
INSERT INTO building_types VALUES (
  UUID(), 'Healthcare Buildings', '[0.20,0.25,0.20,0.20,0.15]',
  '[0.30,0.30,0.15,0.10,0.15]'
);
```

4. Update Config Values

```
UPDATE config SET value = '30' WHERE `key` = 'MAX_OBS';
UPDATE config SET value = '4' WHERE `key` = 'STRENGTH_THRESHOLD';
```

5. Run Tests Before Commit

```
npm run test:automated  
# Expected: "Metadata validation passed" / "Static checks passed" / "All tests passed"
```

6. Debug a Calculation

Send test evaluation via curl → examine `tisByDT`, `cisByAD`, `criteria[0].is` → compare against Excel workbook reference.

7. Database Backup & Restore

```
docker exec mariadb mysqldump -u myuser -pmypassword mydb > backup.sql  
docker exec -i mariadb mysql -u myuser -pmypassword mydb < backup.sql
```

8. Reset Database

```
docker compose down -v && docker compose up -d mariadb && npx ts-node src/scripts/seed.ts
```

12 DEBUGGING GUIDE

Top 5 Errors

1. "DataSource init failed"

Causes: MariaDB not running, wrong DATABASE_URL, port conflict. **Fix:** `docker compose up -d mariadb`; verify credentials match docker-compose.yml.

2. "Metadata validation failed" (exit code 1)

Fix: Run `npx ts-node src/scripts/seed.ts`. Check all 4 core tables have data.

3. "ECONNREFUSED 127.0.0.1:4000"

Fix: Backend not running — `npm start`.

4. CORS Errors in Browser

Fix: Backend uses `cors()` middleware. If frontend port changed, update CORS config in `index.ts`.

5. "Missing configuration: MAX_OBS"

Fix: Re-run seed or insert manually:

```
INSERT INTO config VALUES (UUID(), 'MAX_OBS', '25');
```

Log Locations

COMPONENT	HOW TO VIEW
Backend	Terminal where <code>npm start</code> runs (stdout)
MariaDB	<code>docker compose logs mariadb</code>
Docker	<code>docker compose logs</code>

13 TROUBLESHOOTING

#	PROBLEM	LIKELY FIX
1	Backend won't start	<code>docker compose up -d mariadb</code> then <code>npm start</code>
2	"Access denied for user"	Verify DATABASE_URL matches docker-compose.yml
3	Tables don't exist	<code>npx ts-node src/scripts/seed.ts</code>
4	OBS always 0	Check <code>SELECT * FROM config WHERE key='MAX_OBS'</code>
5	NEB always "No Rating"	Re-seed <code>neb_thresholds</code> table
6	Empty strengths/weaknesses	Check STRENGTH_THRESHOLD & WEAKNESS_THRESHOLD in config
7	"entity.parse.failed"	Invalid JSON in request body — validate payload
8	TypeORM "EntityMetadataNotFound"	Verify all 5 entities in <code>data-source.ts</code> entities array
9	Docker pull fails	Check network; <code>docker pull mariadb:10.11</code> manually
10	TypeScript compilation errors	<code>npm install</code> then <code>npx tsc --noEmit</code>
11	Static test fails in CI	Remove hardcoded values; use DB config
12	Data lost after restart	Verify <code>db_data</code> volume exists: <code>docker volume ls</code>

14 CODING STANDARDS

Core Principles

1. **Metadata-Driven First:** Never hardcode a database-worthy value
2. **Backend Owns Logic:** All calculations & business rules in services
3. **TypeScript Strict:** `strict: true`, no implicit `any`
4. **Single Source of Truth:** Canonical seed data in `src/data/`

Naming Conventions

ELEMENT	CONVENTION	EXAMPLE
Files	kebab-case	<code>calculation-service.ts</code>
Classes/Entities	PascalCase	<code>NebThreshold</code>
Functions/variables	camelCase	<code>getBuildingType()</code>
DB tables/columns	snake_case	<code>neb_thresholds</code> , <code>building_type</code>
API paths	kebab-case	<code>/api/v1/building-types</code>

Service & Controller Rules

- Services export **async functions**, never classes
- Services use metadata service for data access (not direct repos)
- Calculation service is the **only** place formulas exist
- Controllers validate shape before calling services
- Controllers use `try/catch` → 400 for validation, 500 for internal errors
- Success responses always include `ok: true`

Prohibited Patterns (CI-enforced)

```
// WRONG - hardcoded
const weights = [0.2, 0.2, 0.2, 0.2, 0.2];
const NEB = [85, 60, 40];
const MAX_OBS = 25;

// CORRECT - from database
const weights = bt.disability_weights;
const neb = await metadata.getNebThresholds();
const MAX_OBS = await metadata.getConfigNumber('MAX_OBS');
```

Formatting: 2-space indent, no trailing whitespace, files end with newline, semicolons required.

15 GIT WORKFLOW

Branch strategy: `main` (production) ← `develop` (integration) ← `feature/*` / `fix/*` / `chore/*`

Commit format: `<type><scope>: <description>` — types: `feat`, `fix`, `chore`, `docs`, `test`, `refactor`

PR process: Create feature branch → commit → `npm run test:automated` → push → PR to `develop` → CI runs → code review → merge.

CI pipeline (on push to main): test-backend → build-backend → deploy-backend → build-frontend → deploy-frontend (5 jobs, GitHub Actions).

16 DEPLOYMENT

Azure Resources (via `main.bicep`)

RESOURCE	SKU
App Service Plan	B1 Basic Linux
Backend App Service	Node 20 LTS
Static Web App	Standard
MariaDB Server	10.3, TLS 1.2
Application Insights + Log Analytics	Monitoring

Deploy

```
az deployment sub create \  
  --location westeurope \  
  --template-file azure/main.bicep \  
  --parameters dbPassword='<password>' dbAdminPassword='<admin>'
```

Production App Settings

```
PORT=4000  
WEBSITES_PORT=4000  
DATABASE_URL=mysql://myuser:<pwd>@mariadb-aas-  
production.mariadb.database.azure.com:3306/mydb  
DB_TYPE=mariadb  
TYPEORM_SYNCHRONIZE=false
```

17 FAQ

Q: Why TypeORM and not Prisma? A: Mature MariaDB support, entity decorators, built-in migration system, and repository pattern.

Q: Why lazy metadata initialization? A: Defers DB connection to first request — server starts even if DB is temporarily unavailable.

Q: Why derive DT/AD from data instead of config? A: System auto-adapts when criteria are added/removed without code changes.

Q: How do I reset the database? A: `docker compose down -v && docker compose up -d mariadb && npx ts-node src/scripts/seed.ts`

Q: Migrations vs seed? A: Migrations = schema (DDL), seed = data (DML). Migrations create tables; seed populates canonical values.

Q: Can I use PostgreSQL? A: Changing `DB_TYPE` to `postgres` and updating the connection string should work but is untested.

18 USEFUL COMMANDS

COMMAND	PURPOSE
<code>docker compose up -d mariadb</code>	Start MariaDB
<code>docker compose down</code>	Stop MariaDB
<code>docker compose logs -f mariadb</code>	Follow DB logs
<code>docker exec -it mariadb mariadb -u myuser -pmypassword mydb</code>	DB CLI
<code>npm start</code>	Start backend
<code>npm run test:automated</code>	Full test suite
<code>npm run seed</code>	Seed database
<code>curl localhost:4000/health</code>	Health check
<code>curl localhost:4000/api/v1/criteria</code>	Get all criteria
<code>curl localhost:4000/api/v1/building-types</code>	Get building types
<code>curl localhost:4000/api/v1/neb-thresholds</code>	Get NEB thresholds
<code>curl localhost:4000/api/v1/config</code>	Get config
<code>SHOW TABLES;</code>	List DB tables
<code>SELECT COUNT(*) FROM criteria;</code>	Should return 63

19 GLOSSARY

TERM	DEFINITION
AAST	Accessibility Assessment Scheme Tool
AD	Assessment Dimension (1–5): Spatial, Safety, Cognitive, Digital, Social
CIS	Combined Impact Score — weighted sum per AD
DT	Disability Type (1–5): Physical, Sensory, Cognitive, Communication, Multiple
EN 17210	European Standard for built environment accessibility
IS	Impact Score = value × score per criterion
TIS	Total Impact Score = Σ IS per DT
OBS	Overall Building Score = weighted percentage (0–100)
NEB	National Evaluation Benchmark (A+, A, B, No Rating)
MAX_OBS	Maximum possible OBS raw value (25, configurable)
TypeORM	TypeScript ORM for Node.js
Zod	TypeScript-first schema validation
Bicep	Azure infrastructure-as-code language
Seed	Initial data population process
Static Test	Test scanning source code for banned hardcoded patterns